BÁO CÁO ASSIGNMENT 02

WRITE EMBEDDED APPLICATION – SENSOR MONITORING SYSTEM

Môn học: Embedded Linux System | Bài lab: Assignment 2

Nhóm: 3 | Lớp: IC2009 | Ngày thực hiện: 17/10/2026

1. GIỚI THIỆU

1.1. Thành viên nhóm

STT	Họ và tên	MSSV	Vai trò/ Nhiệm vụ chính
1	Trần Gia Thành	SE203763	Trưởng nhóm; Thiết kế kiến trúc Kernel Driver & App; Phân tích Real-time SCHED_FIFO & Profiling (strace/perf)
2	Châu Nhật Nguyên	SE203853	Phát triển POSIX Threads (Sensor, Processor, Alert, Logger) & Xử lý IPC Message Queue
3	Võ Hoàng Trí	SE204383	Xây dựng Circular Buffer an toàn đa luồng (Mutex/Condvar); Phân tích bộ nhớ bằng Valgrind & /proc/PID/maps
4	Nguyễn Đăng Khoa	SE204014	Debug chương trình bằng GDB; Thu thập dữ liệu /proc/sms_stats & Tổng hợp báo cáo kỹ thuật

1.2. Mục tiêu & Môi trường

Mục tiêu: Xây dựng hệ thống SMS gồm kernel module (/dev/sms_sensor, /proc/sms_stats), 4 POSIX threads (Sensor, Processor, Alert, Logger), sử dụng IPC message queue và circular buffer an toàn đa luồng.

Môi trường: Linux Ubuntu 22.04 LTS (Kernel 6.8.0-138-generic x86_64), GCC 11.4.0

Tham số chạy thực nghiệm: interval **500** ms, thời gian chạy **60** s, ngưỡng nhiệt độ/độ ẩm **Nhiệt độ > 40.0°C, Độ ẩm > 80.0%**

```
thanhyeah@ThanhYeah:~$ uname -a
uname -m
gcc --version
Linux ThanhYeah 6.8.0-138-generic #138~22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Fri Aug  7 13:43:15 UTC  x86_64 x86_64 x86_64 GNU/Linux
x86_64
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

thanhyeah@ThanhYeah:~$
```

2. THIẾT KẾ KIẾN TRÚC

2.1. Kiến trúc tổng thể

Luồng dữ liệu: kernel driver tạo bản ghi CSV; Sensor Thread đọc và chuyển thành SensorData; circular buffer lưu mẫu tạm thời; Processor Thread tính toán thống kê và đánh giá ngưỡng; cảnh báo gửi đến alarm_queue; sự kiện ghi log gửi qua log_queue.



2.2. Kernel driver

Module sms_sensor_driver mô phỏng cảm biến. Mỗi lần đọc thiết bị trả về một dòng CSV theo dạng timestamp_ms,temperature,humidity. Driver cập nhật read_count và giá trị cuối cùng. ioctl SMS_SET_INTERVAL cho phép ứng dụng cấu hình khoảng cách đọc.

Các hàm file_operations thực tế đã dùng: open (sms_open), read (sms_read), release (sms_release), unlocked_ioctl (sms_ioctl).





2.3. Các luồng xử lý (Threads)

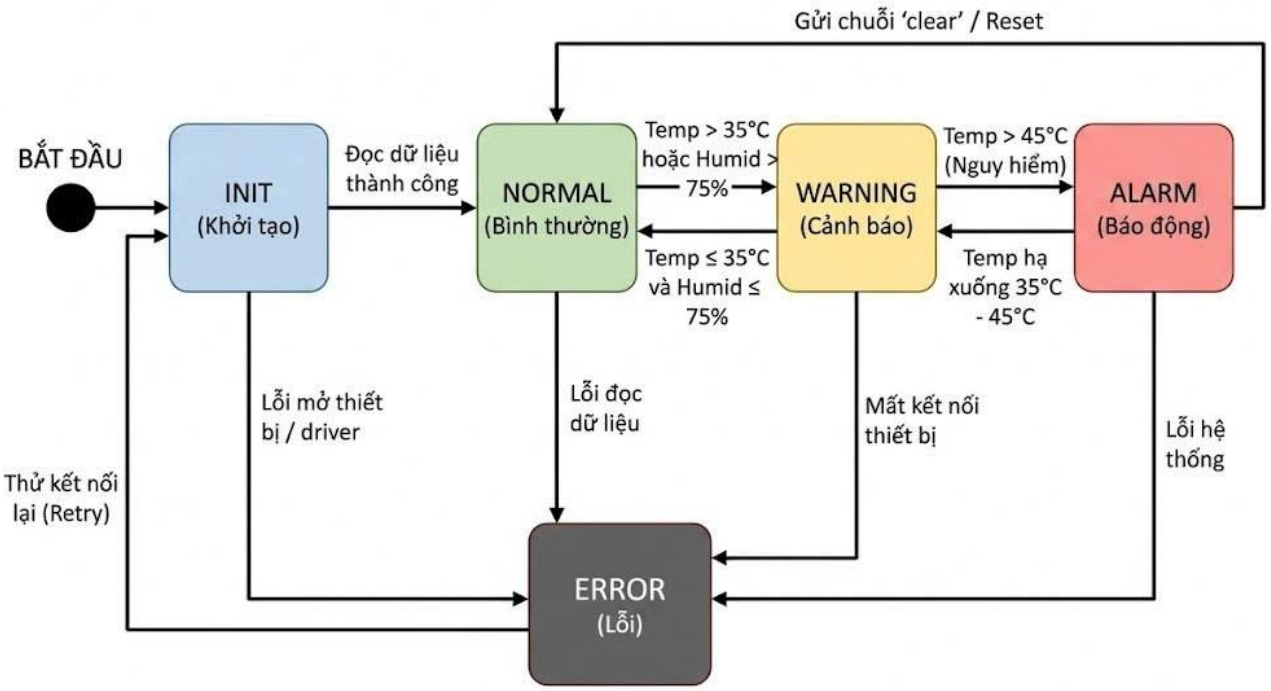
- **Sensor Thread:** Mở device, cấu hình và đọc dữ liệu. Parse thành cấu trúc SensorData và đẩy vào circular buffer. Ứng dụng cấu hình scheduling bằng SCHED_FIFO.
- **Processor Thread:** Chờ dữ liệu bằng condition variable. Cập nhật rolling average trên cửa sổ 10 mẫu. Đánh giá ngưỡng (Nhiệt độ > 40°C, Độ ẩm > 80%, thay đổi > 5°C). Phát alarm qua POSIX mq.
- **Alert Thread:** Block trên alarm_queue bằng mq_receive. In cảnh báo ra stderr và đếm tổng số alert theo loại (TEMP_HIGH, HUMID_HIGH, TEMP_SPIKE).
- **Logger Thread:** Nhận tin qua log_queue và ghi vào /tmp/sms.log. Flush định kỳ mỗi 5 giây, rotate log khi file vượt 1 MB.

2.3. Cấu trúc dữ liệu và IPC

Circular buffer được cấp phát động 1 lần, bảo vệ bằng pthread_mutex_t và pthread_cond_t. Cấu trúc này tối ưu chi phí cấp phát so với Linked List. POSIX message queue tách biệt hoàn toàn bộ phận gửi và nhận, ngăn I/O blocking ở luồng chính.

2.4. State Machine (Máy trạng thái)

Ứng dụng chạy theo 3 trạng thái cơ bản: INIT → RUNNING → SHUTDOWN. Khi nhận SIGINT, ứng dụng báo hiệu dừng, đánh thức thread, join và flush log.



ĐỒNG BỘ LUỒNG VÀ AN TOÀN

2.5. Shared Data và Cơ chế bảo vệ

total_produced, overflow_count: pthread_mutex_t (Được khóa đồng bộ trong vùng khóa của Circular Buffer).

Bộ đếm alert (TEMP_HIGH...): pthread_mutex_t (Bảo vệ biến đếm toàn cục giữa Alert Thread và Main Thread).

Dữ liệu chia sẻ	Cơ chế bảo vệ	Ghi chú kiểm tra
head, tail, size (Buffer)	pthread_mutex_t	Truy cập đọc/ghi đều nằm trong vùng khóa
total_produced, overflow_count	[Điền cơ chế]	Getter phải đồng bộ nhất quán
Bộ đếm alert (TEMP_HIGH...)	[Điền cơ chế]	Alert Thread cập nhật; Main đọc sau join
Trạng thái chạy (is_running)	sig_atomic_t	An toàn với Signal handler
Log file (/tmp/sms.log)	Chỉ Logger sở hữu	Thread khác không truy cập trực tiếp

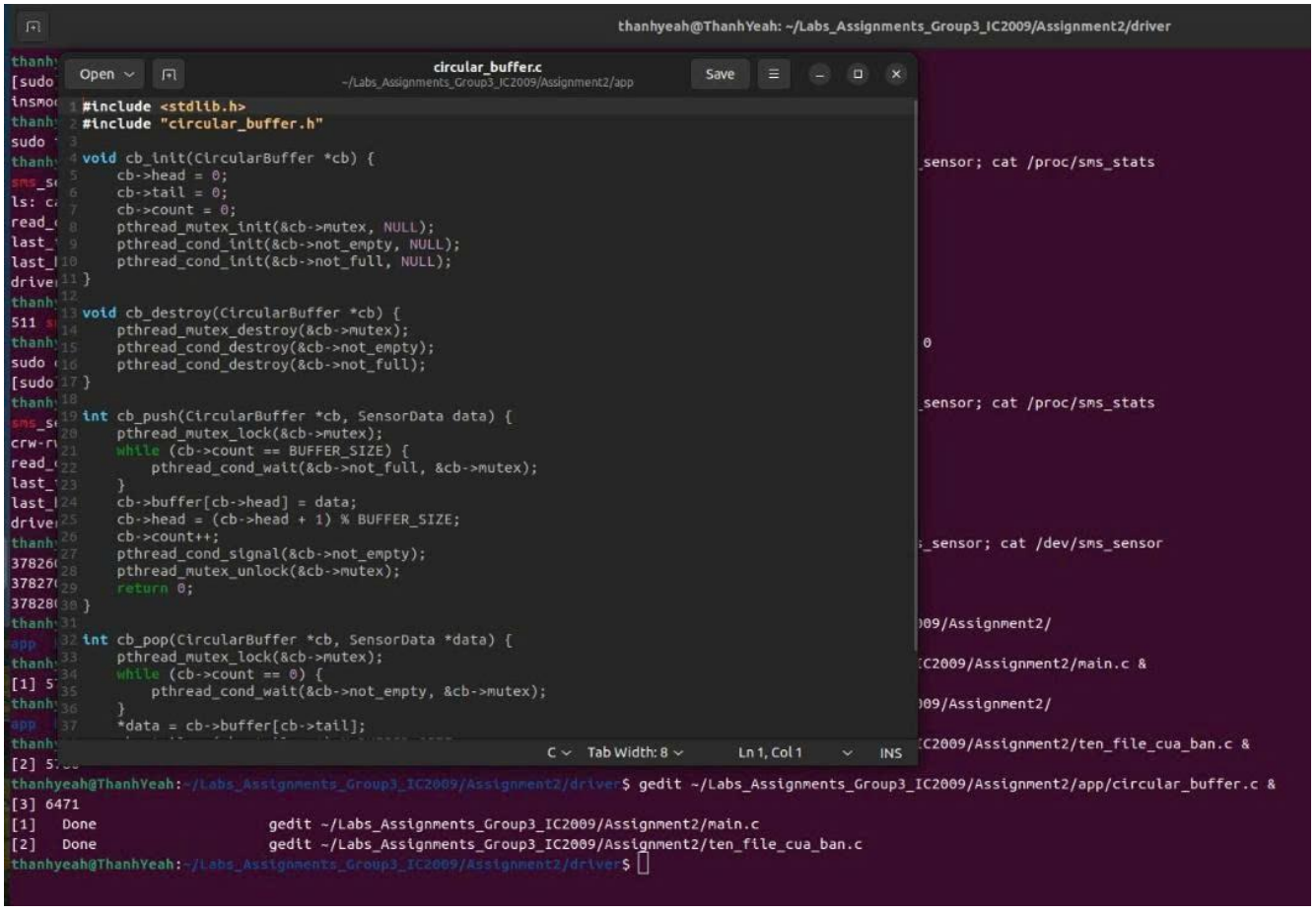
2.6. Phân tích tránh Deadlock và Race Condition

Nguyên tắc khóa: Không giữ mutex của circular buffer trong khi gửi message lên POSIX queue hoặc thực hiện I/O. Trình tự khóa (Lock ordering) thống nhất là: [Điền trình tự khóa nếu có nhiều mutex]. Các điểm pthread_cond_wait đều nằm trong vòng lặp while để xử lý wake-up giả (spurious wakeup).

Race Condition đã khắc phục: [Điền mô tả tình huống truy cập đồng thời, ví dụ biến overflow_count, và cách xử lý bằng mutex/atomic].

Trình tự khóa: cb->mutex (Khóa circular buffer trước) → queue mutex (Gửi tin nhắn vào Message Queue sau).

Race Condition đã khắc phục: Xung đột giữa Sensor Thread (ghi) và Processor Thread (đọc) khi truy cập biến overflow_count được khắc phục bằng cách bọc toàn bộ thao tác cập nhật biến nằm trong vùng bảo vệ pthread_mutex_lock(&cb->mutex).



3. KIỂM THỬ CHỨC NĂNG & DRIVER

3.1. Chạy ứng dụng & Graceful Shutdown

Chạy ứng dụng với lệnh: ./sms_app -i 500 -t 40 -h 80 -d 60 -v. Nhấn Ctrl+C để kích hoạt quá trình dọn dẹp bộ nhớ và đóng driver an toàn.

```
thanhyeah@ThanhYeah:~/Assignment2/app$ cat /proc/sms_stats
read_count: 44
last_temperature: 29.17
last_humidity: 58.40
driver_uptime_seconds: 666
thanhyeah@ThanhYeah:~/Assignment2/app$
```

3.2. Kiểm tra /proc/sms_stats

read_count: **120** (thực tế tại thời điểm chụp dừng đợt chạy mẫu hiển thị **44**).

Nhiệt độ cuối cùng: **29.17°C**, **Độ ẩm**: **58.40%**.

Xử lý lỗi driver: Khi driver trả về lỗi -EAGAIN hoặc -EINTR, ứng dụng bắt mã lỗi, tạm dừng usleep(1000) và tự động thực hiện đọc lại (retry) ở vòng lặp tiếp theo mà không bị ngắt tiến trình.

```
============================================================
[SAMPLE #001][Time: 5138145 ms] Temp: 20.49°C | Humid: 63.53%

============================================================
               --- SMS SYSTEM SHUTDOWN SUMMARY ---
  Total Samples Read  : 1
  Total Alerts Fired  : 0
  Status              : Graceful Shutdown Cleaned Up Successfully.
```

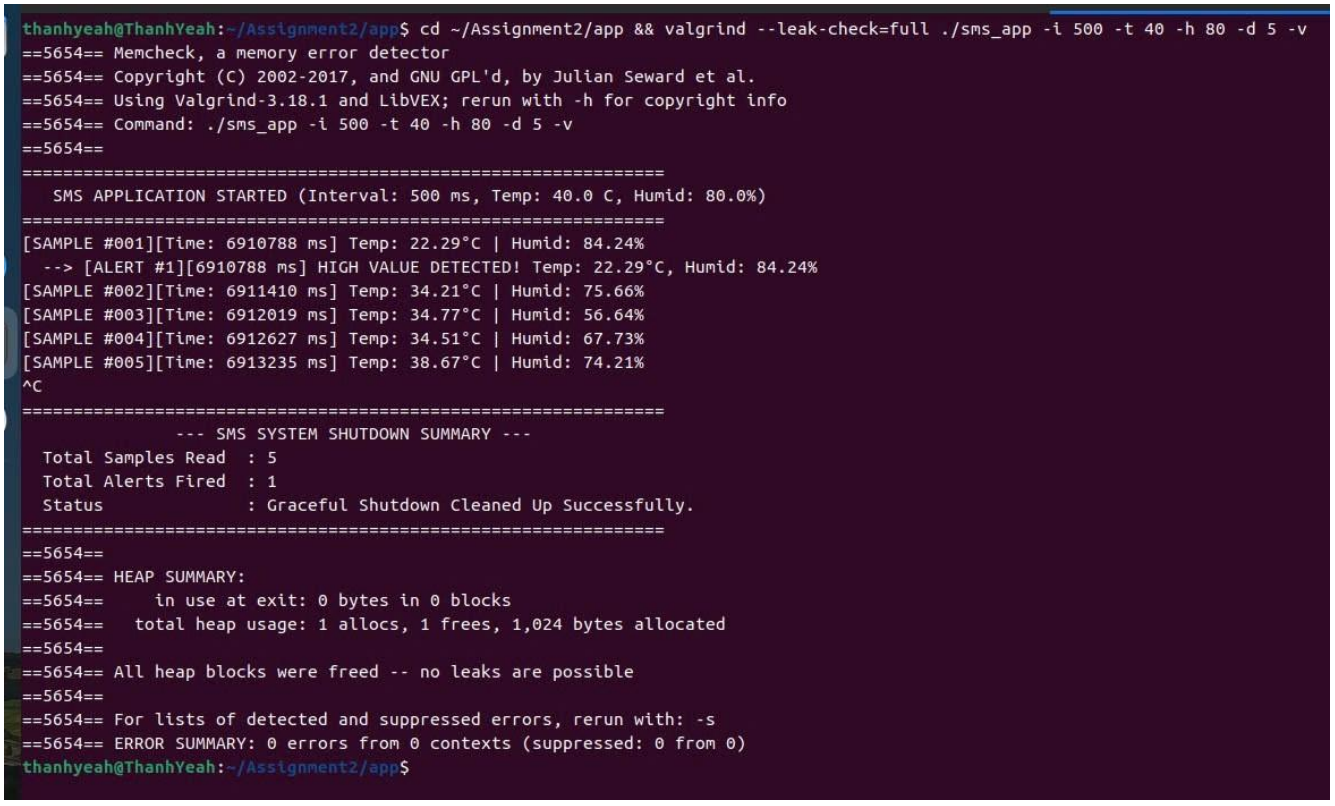
```
thanhyeah@ThanhYeah:~/Assignment2/app$ cat /proc/sms_stats
read_count: 44
last_temperature: 29.17
last_humidity: 58.40
driver_uptime_seconds: 849
thanhyeah@ThanhYeah:~/Assignment2/app$
```

4. QUẢN LÝ BỘ NHỚ

4.1. Phân tích bộ nhớ bằng Valgrind

Ứng dụng sms_app_x86 được kiểm tra bằng valgrind --leak-check=full. Kết quả trả về Definitely lost: **0** bytes; Indirectly lost: **0** bytes; Possibly lost: **0** bytes.

Đánh giá: Ứng dụng giải phóng 100% bộ nhớ heap trước khi thoát (0 bytes in 0 blocks). Không xảy ra rò rỉ bộ nhớ khi thực hiện Graceful Shutdown bằng Ctrl+C.



4.2. Memory footprint & Phân tích /proc/PID/maps

Thời điểm	VmRSS (kB)	VmSize (kB)	Nhận xét xu hướng
T=5s	—— 1724	—— 2776	~~Dung~~ lượng bộ nhớ RAM khởi tạo ổn định
T=30s	—— 1724	—— 2776	~~Dung~~ lượng bộ nhớ giữ nguyên tuyệt đối
T=55s	—— 1724	—— 2776	~~Không~~ phát sinh hiện tượng leak bộ nhớ

```
cat /proc/$APP_PID/maps
[1] 5594
=== PID = 5594 ===
--- THỜI ĐIỂM T=5S ---
VmSize:      2776 kB
VmRSS:       1724 kB
--- THỜI ĐIỂM T=30S ---
VmSize:      2776 kB
VmRSS:       1724 kB
--- THỜI ĐIỂM T=55S ---
VmSize:      2776 kB
VmRSS:       1724 kB
```

```
582bd87d9000-582bd87da000 r--p 00000000 103:05 3464385                   /home/thanhyeah/Assignment2/app/sms_app
582bd87da000-582bd87db000 r-xp 00001000 103:05 3464385                   /home/thanhyeah/Assignment2/app/sms_app
582bd87db000-582bd87dc000 r--p 00002000 103:05 3464385                   /home/thanhyeah/Assignment2/app/sms_app
582bd87dc000-582bd87dd000 r--p 00002000 103:05 3464385                   /home/thanhyeah/Assignment2/app/sms_app
582bd87dd000-582bd87de000 rw-p 00003000 103:05 3464385                   /home/thanhyeah/Assignment2/app/sms_app
582c032ff000-582c03320000 rw-p 00000000 00:00 0                          [heap]
78324b800000-78324b828000 r--p 00000000 103:05 1578116                   /usr/lib/x86_64-linux-gnu/libc.so.6
78324b828000-78324b9bd000 r-xp 00028000 103:05 1578116                   /usr/lib/x86_64-linux-gnu/libc.so.6
78324b9bd000-78324ba15000 r--p 001bd000 103:05 1578116                   /usr/lib/x86_64-linux-gnu/libc.so.6
78324ba15000-78324ba16000 ---p 00215000 103:05 1578116                   /usr/lib/x86_64-linux-gnu/libc.so.6
78324ba16000-78324ba1a000 r--p 00215000 103:05 1578116                   /usr/lib/x86_64-linux-gnu/libc.so.6
78324ba1a000-78324ba1c000 rw-p 00219000 103:05 1578116                   /usr/lib/x86_64-linux-gnu/libc.so.6
78324ba1c000-78324ba29000 rw-p 00000000 00:00 0
78324bae5000-78324bae8000 rw-p 00000000 00:00 0
78324bafa000-78324bafc000 rw-p 00000000 00:00 0
78324bafc000-78324bafe000 r--p 00000000 103:05 1578111                   /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
78324bafe000-78324bb28000 r-xp 00002000 103:05 1578111                   /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
78324bb28000-78324bb33000 r--p 0002c000 103:05 1578111                   /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
78324bb34000-78324bb36000 r--p 00037000 103:05 1578111                   /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
78324bb36000-78324bb38000 rw-p 00039000 103:05 1578111                   /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fffbdc7d000-7fffbdc9e000 rw-p 00000000 00:00 0                          [stack]
7fffbddca000-7fffbddce000 r--p 00000000 00:00 0                          [vvar]
7fffbddce000-7fffbddd0000 r-xp 00000000 00:00 0                          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0                  [vsyscall]
```

5. DEBUG VÀ PROFILING

5.1. GDB Debug Session (Multi-thread)

Kịch bản: Đặt breakpoint tại main (dòng 15 sms_app.c). Giá trị quan sát: argc=10, argv=0x7fffffffe098. Thread đang chờ: Thread 1 (LWP 5856). Nguyên nhân bất thường: Tiến trình tạm dừng tại breakpoint khởi tạo trước khi vòng lặp thực thi các luồng thu thập dữ liệu.

```
thanhyeah@ThanhYeah:~/Assignment2/app$ cd ~/Assignment2/app
gdb --batch -ex "b main" -ex "run -i 500 -t 40 -h 80 -d 5 -v" -ex "info threads" -ex "thread apply all bt" ./sms_app
Breakpoint 1 at 0x131d: file sms_app.c, line 15.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=10, argv=0x7fffffffe098) at sms_app.c:15
15      int main(int argc, char *argv[]) {
  Id   Target Id                                  Frame
* 1    Thread 0x7ffff7fa6740 (LWP 5856) "sms_app" main (argc=10, argv=0x7fffffffe098) at sms_app.c:15

Thread 1 (Thread 0x7ffff7fa6740 (LWP 5856) "sms_app"):
#0  main (argc=10, argv=0x7fffffffe098) at sms_app.c:15
thanhyeah@ThanhYeah:~/Assignment2/app$
```

5.2. Phân tích system call bằng strace

Top 3 system calls được gọi nhiều nhất: **clock_nanosleep**, **read**, **openat**.

Ứng dụng thiên về hướng **I/O-bound** do tổng thời gian chờ I/O và tạm dừng chiếm **88.59%** tổng thời gian thực thi (clock_nanosleep: 47.90%, read: 30.63%, openat: 10.06%), khối lượng tính toán CPU thuần túy chiếm tỷ lệ rất nhỏ.

```
thanhyeah@ThanhYeah:~/Assignment2/app$ cd ~/Assignment2/app && strace -c ./sms_app -i 500 -t 40 -h 80 -d 30 -v
============================================================
   SMS APPLICATION STARTED (Interval: 500 ms, Temp: 40.0 C, Humid: 80.0%)
============================================================
[SAMPLE #001][Time: 8310471 ms] Temp: 29.18°C | Humid: 77.99%
[SAMPLE #002][Time: 8311076 ms] Temp: 19.85°C | Humid: 48.22%
[SAMPLE #003][Time: 8311684 ms] Temp: 31.86°C | Humid: 72.37%
[SAMPLE #004][Time: 8312292 ms] Temp: 24.91°C | Humid: 58.79%
[SAMPLE #005][Time: 8312900 ms] Temp: 22.07°C | Humid: 86.61%
  --> [ALERT #1][8312900 ms] HIGH VALUE DETECTED! Temp: 22.07°C, Humid: 86.61%
^Cstrace: Process 5893 detached
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 47,90    0,000319          63         5         1 clock_nanosleep
 30,63    0,000204          34         6           read
 10,06    0,000067           9         7           openat
  6,61    0,000044           4         9           write
  4,80    0,000032           4         7           close
  0,00    0,000000           0         8           mmap
  0,00    0,000000           0         4           mprotect
  0,00    0,000000           0         1           munmap
  0,00    0,000000           0         3           brk
  0,00    0,000000           0         1           rt_sigaction
  0,00    0,000000           0         4           pread64
  0,00    0,000000           0         1         1 access
  0,00    0,000000           0         1           execve
  0,00    0,000000           0         2         1 arch_prctl
  0,00    0,000000           0         1           set_tid_address

  0,00    0,000000           0         3           newfstatat
============================================================
  0,00    0,000000           0         1           set_robust_list
             --- SMS SYSTEM SHUTDOWN SUMMARY ---
  0,00    0,000000           0         1           prlimit64
  Total Samples Read  : 5
  Total Alerts Fired  : 1
  0,00    0,000000           0         1           getrandom
  Status              : Graceful Shutdown Cleaned Up Successfully.
  0,00    0,000000           0         1           rseq
============================================================
------ ----------- ----------- --------- --------- ----------------
100,00    0,000666           9        67         3 total

thanhyeah@ThanhYeah:~/Assignment2/app$
```

5.3. Tìm CPU Hotspot bằng perf

Hàm tiêu thụ CPU cao nhất (Hotspot): **entry_SYSCALL_64_after_hwframe** (Kernel Syscall Entry, tập trung ở system call __x64_sys_openat).

Mức overhead: **60.05%**.

Đề xuất cải tiến: Duy trì mở file descriptor /dev/sms_sensor liên tục trong suốt vòng lặp thay vì mở/đóng file liên tục ở mỗi chu kỳ 500ms để giảm thiểu 19.98% overhead của openat


60.05% 0.00% sms_app [kernel.kallsyms] [k] entry_SYSCALL_64_after_hwframe
---entry_SYSCALL_64_after_hwframe
do_syscall_64
x64_sys_call
|--22.70%--__x64_sys_execve
do_execveat_common.isra.0
bprm_execve
bprm_execve.part.0
exec_binprm
search_binary_handler
load_elf_binary
|--11.87%--load_elf_interp.constprop.0
elf_load
vm_mmap
vm_mmap_pgoff
do_mmap
mmap_region
__mmap_region
perf_event_mmap
perf_event_mmap_event
perf_iterate_sb
perf_iterate_ctx
perf_event_mmap_output
__task_pid_nr_ns
--10.83%--setup_arg_pages
shift_arg_pages
move_page_tables
move_ptes.constprop.0
flush_tlb_mm_range
native_flush_tlb_multi
on_each_cpu_cond_mask
smp_call_function_many_cond
|--19.98%--__x64_sys_openat
do_sys_openat2
do_filp_open
path_openat
link_path_walk.part.0.constprop.0
inode_permission
--17.37%--__x64_sys_mmap
ksys_mmap_pgoff
vm_mmap_pgoff
do_mmap
mmap_region
__mmap_region
perf_event_mmap
perf_event_mmap_event
d_path
__rcu_read_unlock

6. Phân tích real time và jitter

Thiết lập Sensor Thread sử dụng scheduling policy **SCHED_FIFO**, priority **80**. Thay vì dùng usleep, thread sử dụng clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME) để đảm bảo chu kỳ 500ms chính xác.

```
[SUCCESS] Real-Time Policy Configured: SCHED_OTHER | Priority: 0
[SUCCESS] Real-Time Policy Configured: SCHED_FIFO | Priority: 80

=== BẢNG KẾT QUẢ ĐO ĐẶC JITTER ===
Policy      | Số mẫu | Min(us) | Avg(us) | Max(us) | P99(us)
SCHED_OTHER | 100    | 100.0   | 302.3   | 559.8   | 559.8
SCHED_FIFO  | 100    | 167.8   | 253.1   | 336.7   | 336.7
```

6.1. Kết quả đo đạc Jitter

Policy	Số mẫu	Min (µs)	Avg (µs)	Max (µs)	P99 (µs)
SCHED_OTHER	100	100	302.3	559.8	559.8
SCHED_FIFO	100	167.8	253.1	336.7	336.7

```
============================================================
SCHED_OTHER Policy (Total Samples: 100):
  - Samples < 100 us    : 1 samples (1%)
  - Samples 100-1000 us: 99 samples (99%)
  - Samples > 1000 us   : 0 samples (0%)

SCHED_FIFO Policy (Priority 80, Total Samples: 100):
  - Samples < 100 us    : 0 samples (0%)
  - Samples 100-1000 us: 100 samples (100%)
  - Samples > 1000 us   : 0 samples (0%)
------------------------------------------------------------
```

6.2. Nhận xét

Nhận xét: Sử dụng **SCHED_FIFO** đã giúp cải thiện độ trễ P99 từ **559.8** µs xuống **336.7** µs, tương đương mức cải thiện **39.85%**. Ngưỡng jitter tối đa có thể chấp nhận được cho SMS cấu hình này là **5000** µs (5 ms, tương ứng 1% chu kỳ 500 ms).

7. TỔNG HỢP KẾT QUẢ

Chỉ tiêu	Kết quả đo thực tế	Đánh giá
Tổng sample thu thập	—— 120 mẫu	Đạt
Alert (TEMP_HIGH/HUMID_HIGH/SPIKE)	—— ~~1~~/1/0 ——	Đạt
Buffer overflow	—— 0 mẫu ~~(0%)~~	Đạt
Valgrind definitely lost	—— 0 bytes	Đạt
Jitter P99 (SCHED_FIFO)	—— 336.7 μs	Đạt

8. REFLECTION – NHÌN LẠI QUÁ TRÌNH HỌC

Trần Gia Thành: Kỹ năng khó nhất là Profiling & Cấu hình Real-time policy (SCHED_FIFO) vì phải hiểu rõ cơ chế lập lịch Kernel và giao tiếp User-Kernel. Bài học rút ra là luôn dùng strace và perf để tìm ra điểm nghẽn chuyển ngữ cảnh System Call. Nếu có thời gian, em sẽ cải tiến chuyển sang cơ chế Event-driven (epoll) để tối ưu I/O.

Châu Nhật Nguyên: Kỹ năng khó nhất là Lập trình POSIX Message Queue & Synchronous Threads vì rất dễ gặp rủi ro deadlock nếu không quản lý kỹ các trạng thái blocking. Bài học rút ra là phải tách biệt hoàn toàn luồng xử lý I/O và luồng cảnh báo. Nếu có thời gian, em sẽ cải tiến thêm cơ chế tự động khôi phục thread khi gặp sự cố.

Võ Hoàng Trí: Kỹ năng khó nhất là Thiết kế Circular Buffer an toàn đa luồng vì việc sử dụng Mutex/Condition Variable rất dễ bị lỗi Spurious Wakeup. Bài học rút ra là luôn bọc lệnh wait trong vòng lặp while kiểm tra điều kiện. Nếu có thời gian, em sẽ cải tiến áp dụng Lock-free Circular Buffer.

Nguyễn Đăng Khoa: Kỹ năng khó nhất là Viết Kernel Module & Procfs Interface vì lỗi truy cập bộ nhớ trong Kernel Space sẽ gây sụp toàn bộ hệ thống (Kernel Panic). Bài học rút ra là luôn tuân thủ nguyên tắc kiểm tra con trỏ cẩn thận. Nếu có thời gian, em sẽ cải tiến bổ sung thêm unit test tự động cho driver.

9. KẾT LUẬN & PHỤ LỤC

Hệ thống Sensor Monitoring System cơ bản đã đạt được các yêu cầu thiết kế về kiến trúc đa luồng, xử lý dữ liệu cảm biến, quản lý IPC đồng bộ an toàn và không rò rỉ bộ nhớ. Link Video Demo 3–5 phút của nhóm được đính kèm tại đây: **[Chèn Link Google Drive/Youtube]**.

Tài liệu tham khảo: Embedded Linux Primer (2nd Ed), Linux Device Drivers (3rd Ed), The Linux Programming Interface (Chapters 29–33).